Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3800582

PDF Download
3800582.pdf
06 April 2026
Total Citations: 0
Total Downloads: 165

Published: 05 March 2026
Accepted: 24 February 2026
Revised: 03 January 2026
Received: 18 September 2025

Citation in BibTeX format

RESEARCH-ARTICLE

A Multi-Dimensional Quality Evaluation of Code Generated by LLM-Based Tools Across Multiple Programming Languages

RUIMENG LI, Central China Normal University, Wuhan, Hubei, China

RAN MO, Central China Normal University, Wuhan, Hubei, China

YAO ZENG, Central China Normal University, Wuhan, Hubei, China

ZILI LIU, Central China Normal University, Wuhan, Hubei, China

ZENGYANG LI, Central China Normal University, Wuhan, Hubei, China

YUTAO MA, Central China Normal University, Wuhan, Hubei, China

Open Access Support provided by:

Central China Normal University

A Multi-Dimensional Quality Evaluation of Code Generated by LLM-based Tools Across Multiple Programming Languages

RUIMENG LI, Central China Normal University, China
RAN MO*, Central China Normal University & State Key Lab. for Novel Software Technology, Nanjing University, China
YAO ZENG, Central China Normal University, China
ZILI LIU, Central China Normal University, China
ZENGYANG LI, Central China Normal University, China
YUTAO MA, Central China Normal University, China

Large Language Model (LLM)-based code generation tools have gained widespread adoption in software development. The code quality is an important concern that affects the practical usefulness of generated code suggestions. However, comprehensive evaluations of these tools' code quality remain limited. In this paper, we conduct a systematic comparison of five mainstream code generation tools, including GitHub Copilot, Amazon CodeWhisperer, Tongyi Lingma, Tabnine, and Codeium, across five popular programming languages (C, C++, Java, JavaScript, Python). Using 2,613 programming problems from LeetCode, we generated and analyzed 65,325 code suggestions. We then evaluate their code quality through four dimensions: functional correctness, reliability, maintainability, and complexity. Our experimental results reveal substantial performance variations across tools, languages, and quality dimensions. The overall functional correctness remains limited, with even the best-performing tool achieving only moderate success rates. Reliability and maintainability present additional concerns, with bugs occurring more frequently in hard-difficulty tasks and code smells being pervasive across generated outputs. Although complexity is generally well-controlled, there are still outliers with excessive complexity across all tools. We believe that our study can provide valuable insights into the current state and limitations of LLM-based code generation technology.

CCS Concepts: • **Software and its engineering → Source code generation**.

Additional Key Words and Phrases: Large language models, Code generation, Code quality

1 Introduction

With the rapid development of Large Language Models (LLMs), more and more software developers are using automated code generation techniques [35]. General language models, represented as ChatGPT [40], have shown basic capabilities for generating code based on natural language descriptions of desired functionalities. Subsequently, tools like GitHub Copilot [8], Amazon CodeWhisperer [14], and Codeium [63], etc. have been specifically designed for programming tasks. These tools emerge to enhance the utility of code generation [64].

*Corresponding author

Authors' Contact Information: Ruimeng Li, lrm_ss@email.com, Central China Normal University, China; Ran Mo, moran@ccnu.edu.cn, Central China Normal University & State Key Lab. for Novel Software Technology, Nanjing University, Wuhan, China; Yao Zeng, Central China Normal University, China; Zili Liu, Central China Normal University, China; Zengyang Li, Central China Normal University, China; Yutao Ma, Central China Normal University, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7392/2026/3-ART
https://doi.org/10.1145/3800582

Just Accepted

Although these tools offer considerable convenience to developers, concerns about the quality of the generated code still exist. Especially, the users of these tools include programming beginners and even non-professionals. If they couldn't identify and address potential flaws in the code, this may lead to issues such as frequent logic errors or security vulnerabilities. Eventually, such problems would limit the reliability and applicability of these tools in real-world projects [44].

In recent years, numerous researchers have engaged in in-depth discussions about quality assessment for code generation produced by large language models. They have proposed various evaluation dimensions and standards such as functional correctness, semantic consistency, and maintainability [7]. However, most of the existing studies are limited to a single model or a single programming language. There is a lack of systematic comparisons across languages, models, and multiple evaluation metrics. Furthermore, many existing studies primarily evaluate general-purpose large language models. They generate code by calling model APIs in an offline setting. This is different from how developers use code assistants in their daily work. Such tools often rely on different underlying models and apply targeted engineering optimizations. Therefore, conclusions drawn only from general-purpose models may not be directly generalized to the tools that developers actually use.

To address the above limitations, we conducted a systematic evaluation of five mainstream LLM-based code generation tools across multiple languages and quality dimensions. Unlike studies that mainly evaluate standalone language models such as ChatGPT or base models like GPT in controlled experimental settings, our research focuses on five production-ready code generation tools: GitHub Copilot, Amazon CodeWhisperer, Tongyi Lingma, Tabnine, and Codeium. These tools are actively integrated into development environments and used by millions of programmers daily. They represent the actual systems that developers encounter in practice, complete with tool-specific optimizations, context-aware suggestions, and IDE integration. By focusing on these practical applications, our findings would be directly applicable to developers for tool selection decisions, rather than providing insights only about theoretical model capabilities.

Specifically, we utilized 2,613 programming problems from the LeetCode platform as experimental tasks. These problems cover a wide range of difficulty levels and problem types. On this basis, we evaluated the tools of Copilot, CodeWhisperer, Tongyi Lingma, Tabnine, and Codeium across the five most popular programming languages, including C, C++, Java, JavaScript, and Python. To proceed with our study, we seek to answer the following research questions:

- RQ1: What is the functional correctness of the code generated by different LLM-based tools?
- RQ2: What is the reliability of the code generated by different LLM-based tools?
- RQ3: What is the maintainability of the code generated by different LLM-based tools?
- RQ4: How well do these tools control the complexity of their code suggestions?

Through our comprehensive analysis of 65,325 code suggestions generated across various tool-language combinations, we present several substantial limitations in current tools. Functional correctness remains relatively low, even the best-performing tool achieves only 57.56% average accuracy. Wrong Answer errors are prevalent, accounting for 50-80% of failures. This indicates that although tools can generate syntactically valid code, they struggle with algorithmic reasoning and semantic correctness. Performance varies dramatically across programming languages, with Python (41.91%) and JavaScript (41.75%) outperforming lower-level languages like C (25.66%). Our reliability analysis reveals persistent bug patterns, particularly in Medium-difficulty problems that involve in complex state transitions. Maintainability issues are pervasive, with code smells appearing in nearly all statically-typed language outputs, averaging 1.27-1.46 smells per suggestion. Complexity is generally well-controlled, but outliers with excessive cyclomatic complexity occur across all tools.

These findings demonstrate that current LLM-based code generation tools function most effectively as intelligent coding assistants requiring substantial human oversight rather than autonomous programming solutions.

Just Accepted

Meanwhile, the substantial quality variations across tools, languages, and problem complexities indicate the need for careful tool selection and enhanced quality assurance processes.

We believe this study has made the following contributions to the field of LLM-based code generation and software engineering:

- We provide the first systematic comparison of five mainstream code generation tools across multiple programming languages and quality dimensions, offering practitioners evidence-based insights for tool selection.
- We establish a comprehensive evaluation framework that examines the functional correctness, reliability, maintainability, and complexity of code suggestions generated by LLM-based tools.
- We demonstrate how programming language characteristics substantially influence code generation quality, providing guidance for language-specific deployment strategies.
- We provide a large-scale open dataset. Based on 2,613 coding problems, we create a dataset covering 65,325 code suggestions generated by five LLM-based tools in five popular programming languages. This dataset can serve as a benchmark for future research on code generation performance. In addition, we developed a suite of tools that assist researchers in automatically acquiring problem text, generating code suggestions, and analyzing evaluation results.

The rest of our paper is organized as follows: Section 2 introduces the background; Section 3 reviews the existing work related to this study; Section 4 elaborates our experimental design; Section 5 describes the experimental methods, results, and analysis; Section 6 further discusses the findings and implications; Section 7 presents the possible validity threats; Section 8 concludes the study.

2 Background

2.1 Code Generation

Code generation aims to automatically generate accurate and efficient code from natural language instructions or incomplete code snippets. Typical tasks include inline code completion, context-aware function body prediction, and intelligent code-related question answering. In recent years, various code generation tools have emerged in the industry, including Tongyi Lingma, Tabnine, GitHub Copilot, Amazon CodeWhisperer, and Codeium. Each of them is based on large language models but differs in training methods, deployment strategies, and applications.

Tongyi Lingma [4] is based on Alibaba's Qwen model and integrated with Alibaba Cloud services, creating a cohesive development-to-deployment loop. Tabnine [50], developed by Codota, runs on various platforms and uses a permissive data license, making it suitable for privacy-conscious enterprises. GitHub Copilot [8], a collaboration between GitHub, OpenAI, and Microsoft, is based on the Codex model and trained on open-source code from GitHub, focusing on code completion and contextual prompts. Amazon CodeWhisperer [14] emphasizes context-awareness and code security to enhance developer efficiency while integrating security checks. Codeium [63], developed by Windsurf Inc., offers a large language model that provides context-aware completion suggestions and simplifies access to intelligent programming tools. The tool is now referred to Windsurf, but we retain its original and well-known name in this paper.

We selected the above five code generation tools for this study, since they have a broad user base and are representative in the field. Additionally, these tools are all developed and maintained by companies with strong financial and technical capabilities, ensuring high technical reliability and long-term support. Table 1 reports the basic information of these code generation tools.

2.2 Code quality

Code quality assessment is critical for evaluating the quality of generated code in terms of functional implementation, structural design, and operating characteristics. This assessment plays an important role in improving

Just Accepted

Table 1. Basic Information of Five Code Generation Tools

Tool	Company	Model	Deployment	User Scale
Tongyi Lingma	Alibaba	Qwen	Cloud / Local	Millions
Tabnine	Codota	Proprietary Transformer-based Model	Cloud / Local	Millions
GitHub Copilot	GitHub, OpenAI, Microsoft	Codex (GPT-4)	Cloud	Tens of millions
Amazon CodeWhisperer	Amazon	AWS	Cloud	Millions
Codeium	Windsurf	Proprietary Transformer-based Model	Cloud	Millions

code maintainability, execution efficiency, and overall development quality [48]. In this study, we focus on four key dimensions to evaluate the quality of generated code: 1) Correctness. It refers to whether the generated code meets the expected functions and specified tasks. 2) Reliability. It assesses the code's ability to run stably and consistently, indicating fewer defects and sustained performance. 3) Maintainability. It reflects how easily the code can be understood, modified, and extended. 4) Complexity. It measures the intricacy of the code's structure and control flow. Higher complexity often leads to maintenance difficulties.

2.3 LeetCode

LeetCode [30] is a popular online platform for programming learning and competitions. It supports various programming languages and offers a wide range of coding problems with three difficulty levels: Easy, Medium, and Hard. This variety provides useful test samples for evaluating code generation models [11].

LeetCode also features a standardized input/output format and a stable runtime environment, which ensures consistent evaluation of submissions. Each problem includes test cases to validate user code and provide detailed feedback. Submitted code would result in one of six execution states: 1) Accepted: The code passes in all test cases and satisfies the time and space constraints. 2) Wrong Answer: The code can be compiled and run, but at least one test case produces an incorrect output. 3) Compile Error: The code has syntax errors or is missing dependencies during the compilation phase, thus it cannot enter the running phase. 4) Runtime Error: An error occurred during program execution, causing the program to crash or halt. 5) Time Limit Exceeded: The program runs longer than the time limit set by the platform for some test cases. 6) Memory Limit Exceeded: The program uses more memory than the allowed limits while running.

3 Related work

3.1 Automated Code Generation

Early code generation research mainly relied on heuristic rules and grammar-driven methods [26]. Typically, generation rules are defined based on formal language theory, and the code is automatically constructed through fixed grammar templates. For example, Tree-Adjoining Grammar [27] and Tree-Substitution Grammar [27] [10] were used to synthesize target code together with the rewriting rules of Abstract Syntax Tree (AST). For a long time, code generation research has mainly focused on Domain Specific Languages (DSLs) [13, 20, 25]. After that, researchers started using statistical language modeling [22, 38, 43] for code prediction and completion.

Just Accepted

With the growth of deep learning, researchers applied Convolutional Neural Networks (CNN) [33, 49], Recurrent Neural Networks (RNN) [24, 56] and Long Short-Term Memory Networks (LSTM) [17] into code generation tasks. These methods gradually reduce the reliance on manually written grammatical rules and start to learn the potential relationships between code and natural language for better modeling. Ciniselli et al. [9] systematically evaluated the performance of the BERT model in code completion, verifying its feasibility in structured code generation.

Since the introduction of the Transformer architecture [55], its success in natural language processing has driven its adoption in code generation. Alon et al. [1] proposed the Structured Language Model (SLM), which generates syntactically valid code by modeling its abstract syntax tree (AST). Kim et al. [29] incorporated syntax tree structure information into the Transformer for improving the accuracy of code completion. As training corpora expand and the number of model parameters increases, the Transformer architecture has become the core foundation for building Large Language Models (LLMs). Consequently, code generation has entered a new stage driven by LLMs. [26]. Generalized LLMs such as GPT3 [5, 40], LLaMA series [19, 53], Claude3 [2], and Gemini [51] have shown good code generation capabilities in general-purpose tasks and support multiple programming languages. To meet the higher requirements of code generation tasks, researchers have also developed several specialized and code-oriented LLMs. For example, Codex [8] is a model retrained on large-scale open source code based on the GPT series, which has been widely used in real programming tools such as GitHub Copilot. Nijkamp et al. [39] trained and open-sourced the CodeGen model, which supports multilingual code generation. They further introduced a multi-turn program synthesis mechanism to enhance performance on complex tasks. CodeT5 [62] and CodeT5+ [61] construct a flexible and composable encoder-decoder model framework. By using various pre-training objectives and instruction fine-tuning mechanisms, it achieves leading performance in multiple code understanding and generation tasks. Recently, the StarCoder series [31] [34] and DeepSeek-Coder [21] models have introduced further optimizations in pretraining corpus construction, decoding strategy control, and multilingual adaptability, which bring improved code generation quality. Compared with the general-purpose models, these specialized models are more stable in terms of syntactic correctness, structural rationality, and maintainability of the code. Therefore, they show more practical value in tasks such as IDE programming assistance, unit test generation, and interactive completion.

Although the code generation capabilities of large language models continue to improve, their outputs still exhibit notable issues in terms of reliability and maintainability. Models may generate code with semantic or syntactic errors [47]. Therefore, how to evaluate and improve the quality of code generated by large language models has become an important research direction in the current software engineering field.

3.2 Evaluation of Code Generation Quality

As LLM-based code generation tools have been widely used in software engineering community, researchers have conducted evaluations on their code quality from different perspectives [7].

Functional correctness is the most fundamental quality metric for code generation. It is typically measured by the pass rate of the generated code on predefined test cases. Chen et al. [8] proposed an evaluation dataset HumanEval to measure the Codex's ability in Python code generation tasks. The functional correctness of the generated code is measured using the pass@k metric. Similarly, Austin et al. [3] introduced MBPP and MathQA-Python datasets, which focus on basic programming tasks and mathematical programming tasks, respectively. Cassano et al. [6] extended HumanEval and MBPP into 18 programming languages through automatic translation, resulting in MultiPL-E, the first large-scale multilingual benchmark for code generation evaluation. Mo et al. [36] conducted a large-scale evaluation of GitHub Copilot's functional correctness on LeetCode-based tasks across four programming languages. They also identified 17 common error types, revealing the limitations and underlying causes of Copilot's mistakes across different languages and problem categories. Besides the

Just Accepted

execution-based evaluation using test cases, some studies used compilers or interpreters to examine the success rate in compilation or interpretation. Siddiq et al. [46] proposed the FRANC framework, and Wang et al. [60] introduced COMPCODER, both aim to improve the compilation or interpretation success rate of generated code.

Researchers have also compared the similarities between generated code and reference implementations. Zhang et al. [18] proposed BERTScore,a metric that measures the semantic similarity between candidate and reference texts using contextual embeddings. Zhou et al. [68] extended the idea to code generation tasks by proposing CodeBERTScore. It encodes both natural language input and the generated code itself simultaneously, then considers the semantic consistency between the code and the task context during the evaluation process. Cotroneo et al. [12] proposed a fully automated method called ACCA based on symbolic execution techniques. It can evaluate whether the AI-generated code behaves as a reference implementation. Their research overcomes the limitations of traditional evaluation based on output similarity and lack sufficient semantic coverage. Dong et al. [15] proposed CodeScore, a LLM-based evaluation method. They also introduced UniCE, a unified training framework that measures functional correctness by learning code execution behavior.

Numerous studies focus on evaluating the performance and efficiency of generated code by measuring the time and space complexity during actual execution. Du et al. [16] introduced the Mercury benchmark to assess the operational efficiency of large language models for coding tasks. Huang et al. [23] proposed the EffiBench benchmark, whihc consists of 1,000 efficiency-sensitive LeetCode problems, each paired with a top-performing human reference solution from the LeetCode leaderboard. These references serve as the gold standard for execution time and memory usage. Tuttle et al. [54] evaluated the runtime and energy consumption of eight large language models during LeetCode code generation. All of these studies show that most model-generated answer solutions are generally inferior to human solutions in terms of energy efficiency, reflecting the shortcomings of current models in terms of runtime performance and efficiency.

Security evaluation represents another important research direction in assessing code generation quality for large language models (LLMs). Wang et al. [58] constructed the CodeSecEval dataset, which contains 180 samples covering 44 high-risk security vulnerability types. They demonstrated that many of these vulnerabilities can significantly limit the effectiveness and reliability of LLM-generated code in real development scenarios. Pearce et al. [42] focus on GitHub Copilot and constructed 89 security scenarios based on several high-risk weaknesses from MITRE's CWE Top 25. Using CodeQL combined with manual review to analyze 1,689 Copilot-generated programs, they found that about 40% of code suggestions contained security vulnerabilities. This indicates that a non-trivial proportion of the code generated by Copilot could pose security risks under default usage. Consequently, developers who directly adopt these suggestions may face increased security threats. Yu et al. [66] investigated security from a self-verification perspective. In their experiments, ChatGPT acted as both the developer and the tester. They evaluated its ability to assess the correctness and security of its own code across three types of tasks: code generation, code completion, and program repair. The results showed that ChatGPT is reasonably capable of writing code, but much less reliable in assessing the code it produces. It often misjudges incorrect or vulnerable code, as well as failed repairs, as correct. Even when guided with structured prompts and test-report hints, it still exhibited a high miss rate and produced self-contradictory hallucinations. These studies treat security as an independent quality dimension, and emphasize that developers should remain alert to potential security risks when using large language models for code generation.

Yetistiren et al. [65] used the HumanEval dataset to evaluate the effectiveness, correctness, and efficiency of the code generated by GitHub Copilot. Yetiştiren et al. [64] further evaluated the quality metrics such as functional correctness, code maintainability, security, reliability, and technical debt of ChatGPT, GitHub Copilot, and Amazon CodeWhisperer based on the HumanEval dataset. The results show that ChatGPT has the highest correctness rate (65.2%) and CodeWhisperer generates code with minimal technical debt. D. Wang et al. [57] used LeetCode problems to evaluate GitHub Copilot. They combined actual execution results with SonarQube analysis to assess code reliability, maintainability, and complexity. L. Wang et al. [59] analyzed functional correctness

Just Accepted

and runtime efficiency of large language models based on their Python solutions for LeetCode tasks. Khan et al. [28] examined the code quality of ChatGPT in terms of correctness, readability, and security based on a self-constructed dataset. Florez Muñoz et al. [37] designed ten programming tasks of varying difficulty. They used SonarQube to compare the outputs of eleven AI code generation tools in terms of security, reliability, and maintainability. Yu et al. [67] examined the usage patterns of ChatGPT and Copilot in real-world GitHub projects. They found that the generated code is mostly used for data processing in small projects. Such code tends to be short, low in complexity, and modified less frequently than human-written code. But it often lacks complete records of generation and modification.

Liu et al. [32] studied the quality of 4,066 code suggestions generated by ChatGPT in Python and Java. They examined functional correctness, code quality issues, and iterative refinement strategies. Their study provided valuable insights into how conversational AI models generate and refine code through prompt engineering. However, their work focuses exclusively on ChatGPT, a general-purpose conversational AI model accessed through chat interfaces. Our research evaluates five production IDE-integrated tools (Copilot, CodeWhisperer, Tongyi Lingma, Tabnine, Codeium) to enable evidence-based tool selection for software development workflows. There are several important distinctions between our work and that of Liu et al. [32]: First, we focus on production IDE tools that provide real-time, single-shot code completion rather than conversational AI requiring iterative dialogue, which better reflects the actual developer workflows. Second, our comparative analysis across five tools reveals tool-specific trade-offs and enables optimization for specific contexts. Third, we reveal substantial language-specific performance variations and tool-language interactions. Finally, we systematically quantify relationships between functional correctness, reliability, maintainability, and complexity across tools. Therefore, although both studies observe that LLM-generated code exhibits quality challenges, our specific insights, comparative patterns, and practical implications are fundamentally different.

In summary, existing studies have constructed a variety of evaluation methods and analysis frameworks around multiple dimensions such as functional correctness, semantic similarity, and static quality. However, most of them focus on specific models, specific languages, or single quality dimensions. There is a lack of systematic analysis across multiple models, languages, and dimensions. Some studies cover multiple programming languages but rely on a single model. Others compare the performance of multiple models, but only within a single language or on small-scale datasets. In our study, we examine five mainstream code generation tools, including GitHub Copilot, Amazon CodeWhisperer, Tongyi Lingma, Tabnine, and Codeium, across five popular programming languages (C, C++, Java, JavaScript, Python), and evaluate their code quality through four dimensions: functional correctness, reliability, maintainability, and complexity. We aim to provide a systematic and comprehensive study on the LLM-based code generation tool.

4 Study Design

4.1 Objectives and Research Questions

With the increasing use of large language models for code generation tasks, the quality of the generated code has gained much attention from both academia and industry. However, most current studies primarily focus on functional correctness, but usually neglect other quality dimensions. To bridge this gap, we conduct a comprehensive study on the quality of code generated by LLM-based code generation tools from four key aspects: functional correctness, reliability, maintainability, and complexity. With this goal, we aim to answer the following research questions:

- RQ1: What is the functional correctness of the code generated by different LLM-based tools?
- RQ2: What is the reliability of the code generated by different LLM-based tools?
- RQ3: What is the maintainability of the code generated by different LLM-based tools?
- RQ4: How well do these tools control the complexity of their code suggestions?

Just Accepted

4.2 Tools and Languages

In our study, we leveraged five widely-used LLM-based code generation tools, including Tongyi Lingma, Tabnine, GitHub Copilot, Amazon CodeWhisperer, and Codeium, which are deployed and invoked via Visual Studio Code plugins. Table 2 reports the basics of each tool, including the plugin and version.

Table 2. Code generation tool plugin information

Tool	**Plugin**	**Version**
Tongyi Lingma	Lingma	2.2.0
Tabnine	Tabnine	3.225.0
GitHub Copilot	GitHub Copilot	v1.99.3
Amazon CodeWhisperer	Amazon Q	1.48.0
Codeium	Windsurf	1.40.1

We chose five programming languages: C, C++, Java, JavaScript, and Python, as the focus for code generation and evaluation. First, regarding language characteristics, these five languages represent a wide range of programming paradigms and execution mechanisms. They include both statically typed languages (C, C++, Java) and dynamically typed languages (JavaScript, Python), both compiled languages (C, C++, Java) and interpreted languages (JavaScript, Python). This diverse selection helps evaluate the cross-language generalization and adaptability of LLM-based code suggestion tools. Second, these languages are commonly used and hold a dominant position in the software community. According to the TIOBE Programming Language Ranking [52], C, C++, Java, Python, and JavaScript have consistently ranked among the most popular languages over the past two years.

4.3 Experimental environment

To ensure the stability and consistency of the code generation and quality assessment experiments, we complete all the processes, including code generation, quality inspection, and data analysis, in a unified local environment. The experimental environment is configured as follows:

Hardware environment	CPU: 13th Gen Intel(R) Core(TM) i7-13650HX Memory: 16GB Operating system: Windows 11, 23H2
Software environment	Visual Studio Code: Version 1.99.3 LeetCode Platform: Web-based LeetCode (https://leetcode.com) SonarQube: version 10.7.0.96327, deployed on local server Cppcheck: version 2.16.0 Python: version 3.10

4.4 Experimental Procedures

Figure 1 depicts the overview of our experiments. First, we collected coding problems from LeetCode. In this step, we implemented Python scripts that automatically call the public API provided by LeetCode to extract coding problems. For each problem, our scripts extract three important pieces of information: 1) the problem

Just Accepted

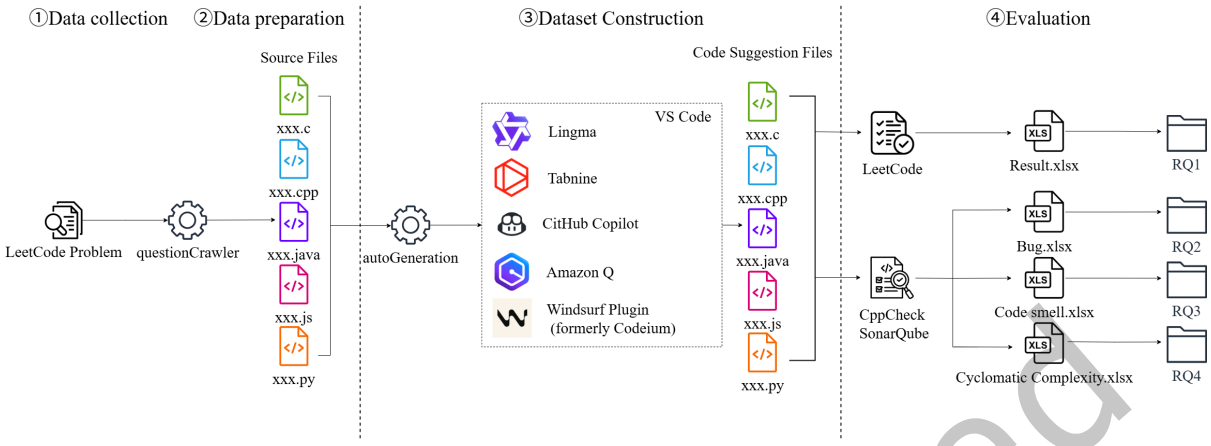

Fig. 1. Experiment Procedure

Just Accepted

description, expressed in natural language, which outlines the background, input/output formats, and relevant constraints; 2) the function template, which specifies the function name, parameter types, and return type used by the online judge; and 3) the difficulty label (Easy, Medium, Hard) assigned by LeetCode. The function templates help standardize the structure of the generated code, and the difficulty labels would allow us to stratify the results in our subsequent analyses. This enables us to examine how tool performance changes as problem difficulty increases. Specifically, we report evaluation metrics at three difficulty levels: Easy, Medium, and Hard. This helps more clearly demonstrate how increasing problem difficulty affects each metric and reduces the bias caused by the unbalanced number of problems across difficulty levels. Meanwhile, practitioners can identify which tools maintain acceptable quality at their typical task complexity level, rather than relying on overall averages that may not reflect their use case.

To ensure fair cross-language comparisons and consistent evaluation, we applied systematic filtering criteria to retain only problems that satisfy all of the following requirements: 1) The problem must be non-premium (i.e., accessible without a paid LeetCode subscription) to ensure our dataset can be reproduced by other researchers; 2) LeetCode should provide complete and well-defined function templates in all five target languages (C, C++, Java, JavaScript, and Python), ensuring structural consistency across languages; 3) The problem description must explicitly specify input/output formats, constraints, and expected behavior without ambiguity; 4) The problem shouldn't require interactive input/output or depend on external systems (e.g., databases, file systems, network services), as these may introduce variability beyond code generation quality. Finally, we collected a total of 2,613 problems in different domains and difficulty levels.

Second, we prepared the data for subsequent experiments by categorizing all problems by programming language and difficulty level. We formatted each problem description into comments according to the syntax conventions of the respective programming language. Following that, we wrote the commented problem descriptions along with the corresponding language-specific function templates into standardized code files. These files would be saved in .c, .cpp, .java, .js, and .py formats, which provide a unified and well-structured input for the code generation tools.

Third, we used LLM-based tools to automatically generate code suggestions. Visual Studio Code (VS Code) is a widely used integrated development environment that supports various programming languages and has excellent plugin expansion capabilities. This makes it a suitable platform for multi-model code generation experiments. We deployed five plugins for LLM-based code generation tools in VS Code: Lingma, Tabnine, GitHub Copilot,

Amazon CodeWhisperer, and Codeium. All plugins were configured with default settings to ensure consistent experimental conditions and reproducibility of results. In this case, we developed a Python script to facilitate batch collection of code suggestions. This script first analyzed the structure of each code file to locate the empty function body based on the syntax rules specific to each language. Then it triggered the code completion feature of each plugin through the editor interface, accepted the first suggestion returned, and saved the result.

Finally, we evaluated the generated code suggestions through both functional testing and static analysis. For functional correctness assessment, each code suggestion was submitted to the LeetCode platform to determine acceptance status (i.e., whether the code successfully passed all test cases). We measured functional correctness as the proportion of problems for which the generated code was accepted. Additionally, we used CppCheck and SonarQube to scan the generated code to measure its reliability, maintainability, and complexity: *Bug*. It reflects the potential defect in code, such as logic loopholes, null pointers, or uninitialized variables, etc. A higher number of bugs often indicates greater unreliability and instability, while fewer bugs suggest more robust and reliable code. *Code Smell.* It refers to an issue that doesn't cause immediate program failures but may compromise readability, extensibility, and maintainability [41]. A higher number of code smells indicates poorer structural quality and higher maintenance cost, while fewer smells suggest that the generated code adheres to better standards and is easier to maintain. *Cyclomatic Complexity.* It reflects the number of independent paths in the code. The higher the value is, the more logical branches there are, and the more difficult it is to test and modify.

To ensure the reliability of the static analysis results, we manually verified the bugs and code smells reported by SonarQube and Cppcheck. Specifically, we grouped the identified issues based on the combination of code generation tool, programming language, and problem difficulty. For each group, we randomly sampled 4 issues. If a group contained fewer than 4 issues, we included all of them. In total, we sampled and validated 274 bug issues and 300 code smell issues. For each sampled issue, the annotators assigned one of four labels: *Confirmed*, *Likely*, *False Positive*, or *Unclear*. *Confirmed* means that the issue clearly corresponds to a real bug or code smell within the context of the current code and problem description; *Likely* means that the issue is reasonable at the level of rule definition, but has limited impact under the constraints of the current problem. *False Positive* means that, after considering both the problem constraints and the code logic, the issue can be judged as not valid. *Unclear* is used for situations where a reliable judgment cannot be made. The first and third authors independently labeled the issues. For the ones with disagreements, the participants would discuss to achieve a consensus. After the independent labeling, the agreement rate between the two annotators was 91%, and the Cohen's kappa coefficient under a binary mapping (treating only Confirmed as *correct* and all other labels as *non-confirmed*) was 0.82. This indicates our validation process has a relatively high inter-rater reliability.

Data Availability: We have shared our collected dataset, including program problems, code suggestions across tools and languages, and analysis results on our website:
https://github.com/lrmsspace/LLM-CodeQualityEval

5 Experimental Results and Analysis

5.1 Analysis of Functional Correctness

5.1.1 ***Correctness Analysis.*** Figure 2 presents a heatmap showing the functional correctness of code generated by five LLM-based tools across five programming languages. The results reveal substantial variations in performance both across tools and programming languages.

Tool-specific performance. Among the five tools, CodeWhisperer outperforms the others with the highest average correctness rate of 57.56% across all languages. This tool consistently achieves functional correctness above 59% in four out of five languages, with particularly strong performance in C++ (64.22%) and Python (61.65%). GitHub Copilot ranks second with an average correctness of 45.82%, showing relatively balanced performance across languages with correctness rates ranging from 35.78% in C to 52.85% in JavaScript. Codeium achieves moderate

Just Accepted

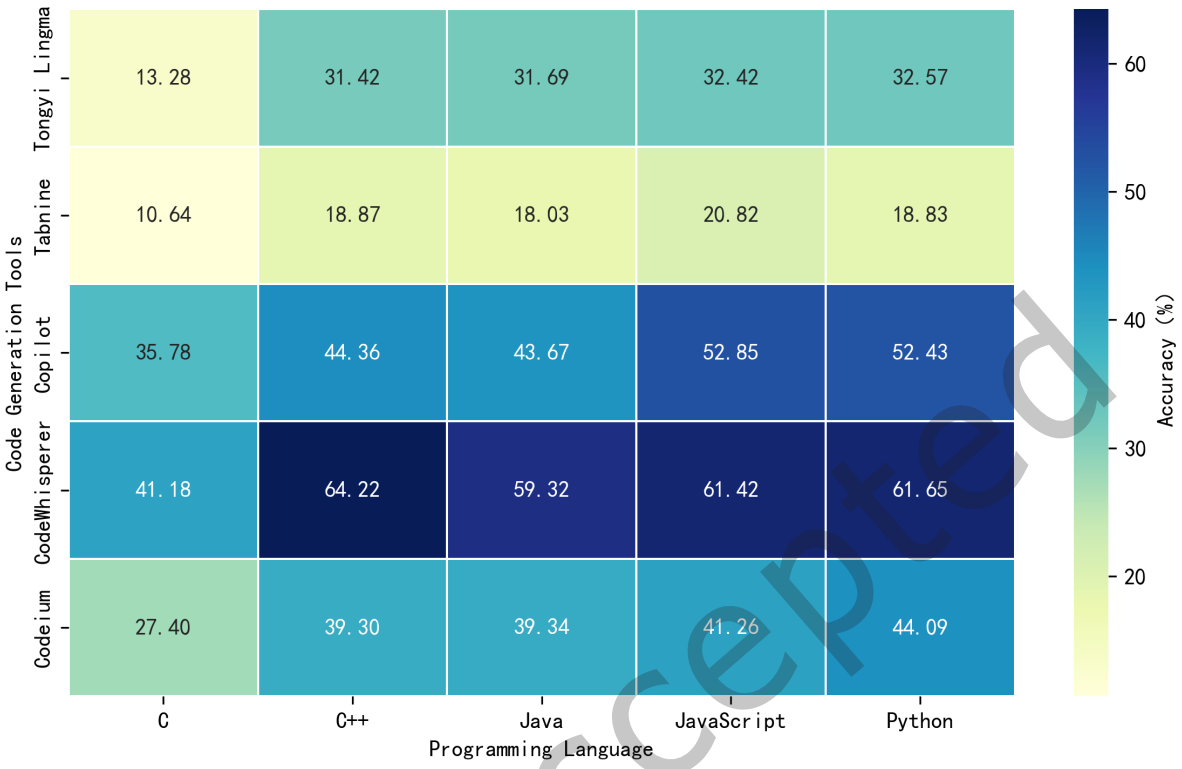

Fig. 2. Heatmap of the correctness of five tools across different programming languages.

Just Accepted

performance with an average correctness of 38.28%, displaying consistent results across most languages but with notable weakness in C (27.40%). Tabnine shows poor overall performance with only 17.44% average correctness, struggling particularly with C (10.64%). Tongyi Lingma exhibits highly variable performance, achieving 32.57% and 32.41% in Python and JavaScript, respectively, but performing poorly in others, particularly in C (13.28%).

Language-specific patterns. The analysis reveals distinct performance patterns across programming languages. Python emerges as the most favorable language for code generation, achieving the highest average correctness rate of 41.91% across all tools. This likely stems from Python's readable syntax, extensive standard library, and the prevalence of Python code in training datasets. JavaScript follows closely with a 41.75% average correctness rate, benefiting from its flexible syntax and forgiving runtime environment. Java demonstrates moderate performance with a 38.41% average correctness rate, though this varies considerably across tools. C++ shows mixed results, with an average of 39.63%, as some tools perform well while others struggle with its complex syntax and memory management requirements. C presents the greatest challenge for all tools, achieving only a 25.66% average correctness rate. This poor performance reflects the inherent complexity of C programming, including manual memory management, pointer arithmetic, and strict compilation requirements that demand precise syntax adherence.

Tool-language interaction analysis. We can observe that tool performance is not uniform across languages. The heatmap reveals several notable patterns in the intersection of tools and languages. CodeWhisperer exhibits particularly strong performance in C++ (64.22%), Java (59.32%), and C (41.18%), substantially outperforming

other tools in these languages. The difference is especially large in C, where CodeWhisperer achieves 41.18%, which is almost 4 times higher than the performance of Tabnine (10.64%). This suggests that CodeWhisperer's training or architecture is better suited for handling strict type systems, explicit memory management, and compilation requirements. Copilot maintains relatively consistent performance across all languages (ranging from 35.78% in C to 52.85% in JavaScript), without extreme strengths or weaknesses. Although it may not achieve the best performance in any single language, this consistency makes Copilot a feasible option for development teams working across multiple languages. Codeium and Tongyi Lingma show better performance in Python and JavaScript compared to C/C++. Tongyi Lingma achieves 32.57% in Python but only 13.28% in C. This pattern suggests that these tools may be optimized for higher-level, dynamically-typed languages commonly used in web development and data science contexts. These interaction patterns indicate that tool selection could be guided by the primary programming languages used in a project rather than the overall reputation of the tools.

The overall functional correctness results are concerning for practical deployment, with the best-performing tool achieving only 57.56% average correctness. This indicates that current LLM-based code generation tools require significant human oversight and validation before deployment in production environments. The wide performance variations both across tools (ranging from 17.44% to 57.56%) and languages (ranging from 25.66% to 41.91%) suggest that tool selection should be carefully considered based on the target programming language and application domain.

These findings present the current limitations of LLM-based code generation technology and highlight the need for ongoing research and development to enhance reliability and consistency across various programming paradigms and complexity levels.

Furthermore, we analyzed the acceptance rate of code suggestions generated by each tool across three levels of problem difficulty. Figure 3 presents the experimental results. Each bar denotes one code generation tool, and the stacked segments indicate the proportion of accepted solutions at different difficulty levels. Using Tongyi Lingma in C as an example, the acceptance rate for Easy problems is 34.71%. For Medium problems, the rate declines to 8.34%. For Hard problems, it further decreases to 1.56%. According to the figure, we can see that all tools demonstrate a clear inverse relationship between problem difficulty and functional correctness, with acceptance rates consistently declining as problem complexity increases from Easy to Hard. This pattern is universal across all tool-language combinations, suggesting a fundamental limitation in current models' ability to handle complex algorithmic reasoning and intricate implementation details. For Easy problems, most tools achieve relatively reasonable performance, with several tool-language combinations exceeding 60% acceptance rates. However, this performance deteriorates significantly for Medium problems, with most combinations falling to 30-50% acceptance rates. Hard problems present the greatest challenge, with many tool-language combinations achieving acceptance rates less than 30%, and some falling below 10%.

The difficulty-dependent performance patterns have noticeable implications for the real-world deployment of code generation tools. While these tools may be suitable for assisting with straightforward, well-defined programming tasks (analogous to Easy problems), their reliability decreases substantially for complex algorithmic challenges or intricate implementation requirements. This suggests that current tools are most effective as coding assistants for routine tasks rather than autonomous solutions for complex programming challenges, and that human oversight becomes increasingly critical as task complexity increases.

5.1.2 ***Error Type Analysis.*** To understand the possible causes of failures, we analyze error records of the "Not Accepted" code suggestions on LeetCode. The errors have five types: Wrong Answer (WA), Compilation Error (CE), Runtime Error (RE), Time Limit Exceeded (TLE), and Memory Limit Exceeded (MLE). Figure 4 shows how different types of execution failures are distributed in the generated code. For example, in the C results, Copilot produces 36.59% Wrong Answers, 14.24% Compilation Errors, 8.38% Runtime Errors, 4.63% Time Limit Exceeded cases, and 0.23% Memory Limit Exceeded cases.

Just Accepted

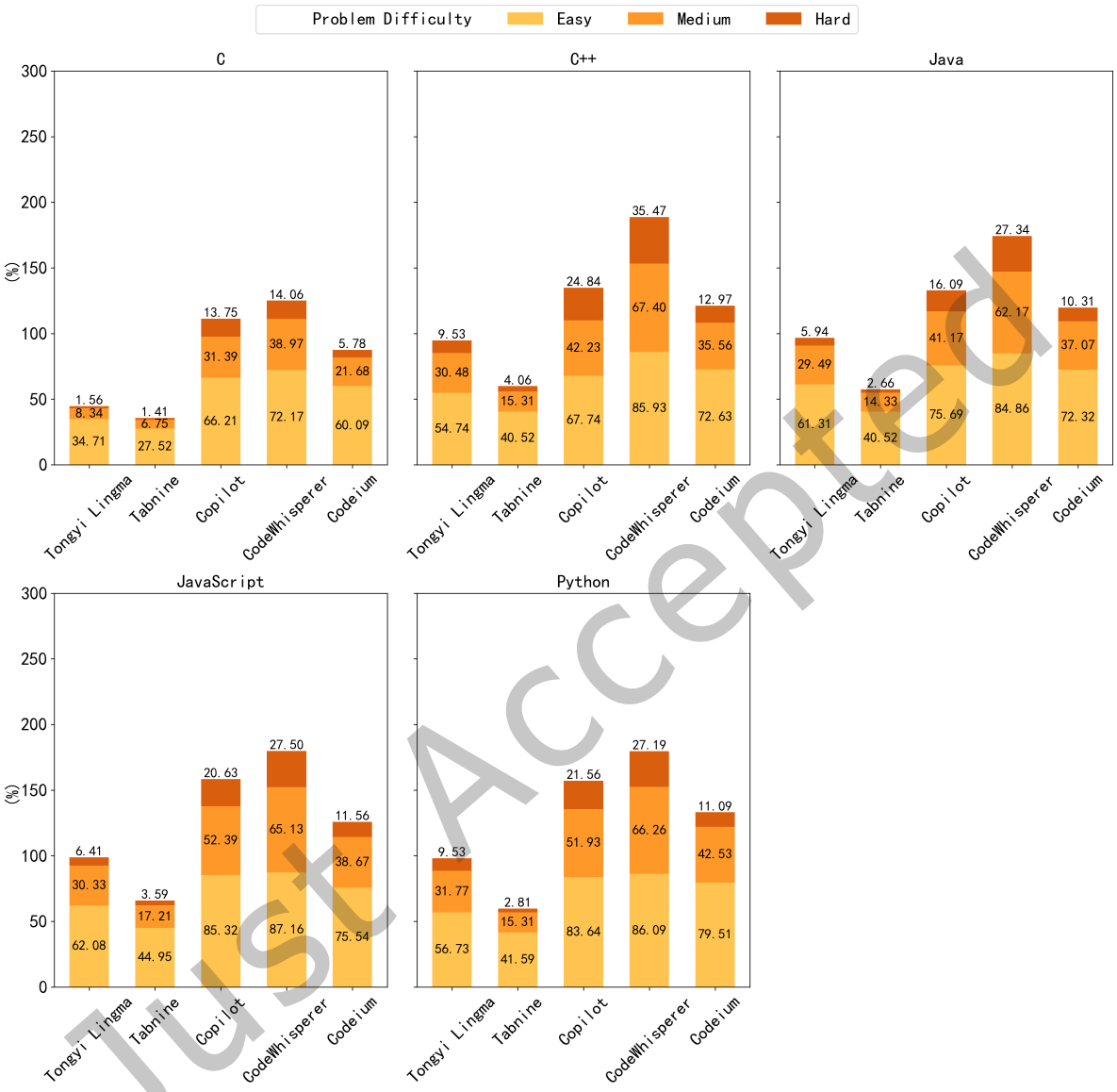

Fig. 3. Acceptance Rate of Code Suggestions by Problem Difficulty

We can observe that Wrong Answer (WA) constitutes the dominant error category across all tool-language combinations, accounting for approximately 50-80% of all failures. This pattern indicates that although most generated code compiles and executes without crashing, the primary challenge lies in implementing correct algorithmic logic to produce the expected outputs. The prevalence of WA errors suggests that current LLMs can generate syntactically valid code structures but struggle with semantic correctness and algorithmic reasoning.

Just Accepted

Furthermore, we examine the distribution of wrong answers based on problem type. We classified all coding problems into three high-level types based on their primary computational characteristics: algorithmic, state-based, and mathematical. *Algorithmic problems* mainly consist of standard data structure and algorithm tasks involving arrays, strings, linked lists, trees, graphs, sorting, sliding window, two-pointer techniques, and similar classical algorithmic patterns. *State-based problems* focus on tasks that require explicit design of states and transition relations, such as dynamic programming, state compression, and game problems. *Mathematical problems* cover the tasks involving number theory, combinatorial counting, probability and statistics, geometry, and other tasks that rely more on symbolic reasoning and formula derivation. Among the 2,613 coding problems, 59.6% are algorithmic, state-based and mathematical problems account for 20.5% and 19.8%, respectively.

From the perspective of problem types, we can observe several distinct patterns of the WA error distributions. Among all the WA errors produced by the five tools, algorithmic problems account for 55.5%, which is about 7% lower than their 59.6% representation in the dataset. State-based problems comprise 20.5% of the dataset, but contribute 25.2% of WA errors, representing a 23% increase. Mathematical problems show nearly proportional representation (19.3% of WA errors vs. 19.8% of the dataset). These patterns indicate that state-based problems present greater challenges for current code generation tools, while algorithmic and mathematical problems are handled more reliably with respect to their prevalence. This disparity likely stems from fundamental differences in problem-solving requirements. Algorithmic problems often follow recognizable patterns (such as depth-first search for tree traversal, or sliding window for substring problems) that models can learn through pattern matching on training data. Mathematical problems typically require domain knowledge, but they usually have deterministic solution paths once the correct formula or theorem is identified. In contrast, state-based problems often demand multi-step reasoning, which includes: 1) identifying appropriate state representations from problem descriptions; 2) determining which state variables are necessary and sufficient; 3) defining valid state transitions; and 4) establishing correct base cases and termination conditions. This complex reasoning chain, often requiring creative insight rather than template application, potentially exceeds the capabilities of current models that are primarily trained on code patterns. These observations could suggest concrete directions for improving code generation models. The disproportionate WA errors in state-based problems indicate that enhancing abstract reasoning and state modeling capabilities should be a priority for future model development.

Compilation Errors (CE) represent the second most frequent error type, particularly prominent in statically typed languages such as C, C++, and Java. These errors typically stem from syntax violations, type mismatches, undeclared variables, or missing imports. The higher frequency of CE in compiled languages reflects the stricter syntactic and semantic requirements imposed by their type systems compared to dynamically typed languages like Python and JavaScript.

Runtime Errors (RE) occur less frequently but indicate critical issues such as null pointers, array bounds violations, or division by zero. These errors are particularly concerning since they represent code that compiles successfully but fails during execution, potentially leading to system crashes in production environments. Time Limit Exceeded (TLE) and Memory Limit Exceeded (MLE) errors are relatively rare but meaningful, as they indicate algorithmic inefficiency or inappropriate resource usage.

For the language-specific analysis, we can find that C shows the highest proportion of compilation and runtime errors, which reflects the complexity of memory management and pointer arithmetic. Java and C++ exhibit moderate error rates, while Python and JavaScript show a higher proportion of wrong-answer errors but no compilation failures, consistent with their interpreted nature and dynamically typed systems.

For the tool-specific analysis, we can see that CodeWhisperer and Copilot show more balanced error distributions with fewer compilation errors, which suggests better syntax handling capabilities. In opposite, Tongyi Lingma exhibits higher proportions of compilation and runtime errors, particularly in C/C++. Tabnine and Codeium demonstrate intermediate patterns, with Codeium showing slightly better error distribution balance across languages.

Just Accepted

These error patterns reveal that current LLM-based code generation tools still face distinct challenges across different programming paradigms. Especially for lower-level languages, they require a more sophisticated understanding of system-level constraints and memory management principles.

Fig. 4. Distribution of error types in the code generated by each tool-language combination

5.2 RQ2: Analysis of Reliability

In this analysis, we investigate the bugs present in code suggestions to evaluate their reliability. According to the results from our manual verification, we estimate that static analysis tools in this study have a confirmed bug issue reporting rate of 74.8% (95% CI: [69.4%, 79.6%]) and a false positive rate of 7.3%. This indicates that the static

Just Accepted

analysis findings can be used as a reliable basis for our analysis on reliability. A lower number of bugs generally indicates more robust and reliable code. Figure 5 depicts the distribution of buggy code suggestions generated by the five tools across different programming languages and difficulty levels. For example, in Tongyi Lingma's code suggestions for Easy problems, we observe 40 buggy suggestions in C, 5 in C++, 29 in Java, 21 in JavaScript, and 11 in Python. From the figure, we can obtain the following observations.

For tool-specific analysis, Codeium demonstrates the strongest overall reliability, which produces the fewest bugs across all language-difficulty combinations. This tool consistently generates code with minimal bugs. CodeWhisperer follows as the second-best tool in terms of reliability, which shows particularly strong performance in statically typed languages (C, C++, Java). Its enterprise-focused training appears to provide advantages in handling complex syntax and memory management requirements. GitHub Copilot exhibits moderate reliability, with relatively balanced bug distributions across languages, although it shows some weakness in handling complex boundary conditions and state transitions. Tabnine's reliability performance varies, showing challenges in C and C++ where pointer arithmetic and memory management introduce additional complexity. Tongyi Lingma shows the most inconsistent reliability, with variations across both languages and difficulty levels, which suggests less robust training on diverse programming scenarios.

For cross-language analysis, the results show that Java exhibits the highest bug rate, reaching 15.71%. In Java code suggestions, Tabnine produces 589 total bugs (93 for Easy, 307 for Medium, and 189 for Hard problems), followed by Copilot with 571 buggy suggestions (71 Easy, 272 Medium, 228 Hard). C code suggestions show a lower bug rate of 4.26%, but many issues are concentrated in high-risk areas such as memory management and pointer manipulation. Reflecting that current large language models still face considerable challenges in handling low-level language syntax, memory management, and structural constraints. Common bugs in C and C++ code suggestions frequently result from integer overflows, which are reported by Cppcheck under the rule name `integerOverflow`. This issue occurs when a numerical operation produces a value that exceeds the range allowed by the underlying integer type. Due to the lack of built-in overflow checks in C and C++, these bugs are common but difficult to detect.

In contrast, Python demonstrates the highest reliability, with fewer buggy outputs across all tools. For instance, Codeium produced only 4 buggy suggestions in Python, clearly outperforming its results in other languages. JavaScript exhibits a moderate bug rate but still shows recurring structural issues across tools, including unsafe or imprecise conditional logic (`javascript_S2871`) and redundant assignments or ineffective statements `javascript_S3500`). Figure 6 presents a case where the model uses `.sort()` without providing a custom comparison function. This leads to unreliable alphabetical sorting in JavaScript due to locale-dependent string comparison behavior. Figure 7 shows an attempt to modify a variable declared with `const`, which violates JavaScript's constraint that `const` variables cannot be reassigned after initialization.

From a tool-language perspective, several combinations require special attention due to elevated bug rates. Tabnine-Java has the highest bug count (589 bugs), followed by Copilot-Java (571 bugs). These two pairings are particularly unreliable and unsuitable for production use without intensive review. Similarly, Tongyi Lingma-C also raises concerns. It encounters 195 bugs, with 40 bugs in Easy problems alone, which indicates fundamental difficulties with C's low-level requirements. These combinations suggest that certain tools may lack the specialized training or architectural features necessary for specific language paradigms. On the other hand, some tool-language combinations present stronger reliability. Codeium-Python has only 4 total bugs, representing the most reliable single combination in our study. Codeium-Java, despite Java's general challenges, shows substantially better reliability than other tools. Additionally, Tabnine-C++ and CodeWhisperer-C++ both demonstrate effective handling of C++ features, resulting in low bug counts. These combinations represent safer choices when reliability is critical.

Just Accepted

All tools generate the fewest buggy code suggestions in easy problems. However, even for Easy problems, none of the tools can make sure all generated code suggestions are bug-free. Code suggestions for Medium and Hard problems more frequently involve reliability issues.

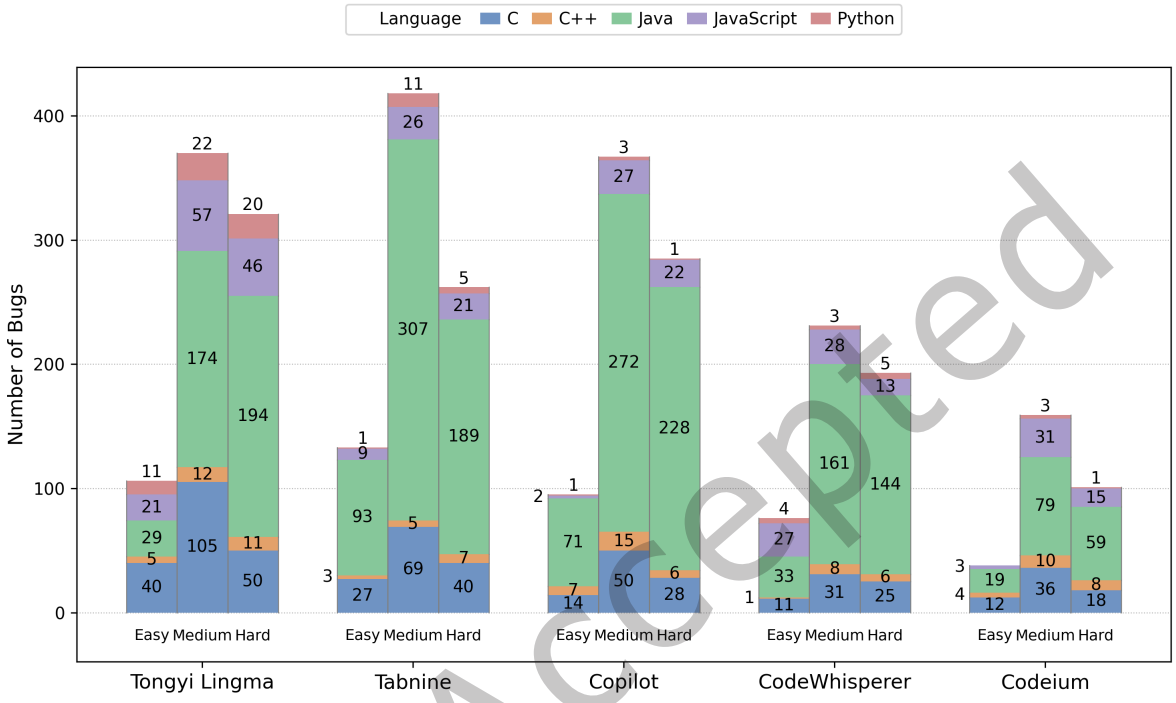

Fig. 5. Number of buggy code suggestions by tools in different programming languages across difficulty levels

```
var isScramble = function(s1, s2) {
    if (s1 === s2) return true;
    const arr1 = s1.split("").sort();
```

Provide a compare function that depends on "String.localeCompare", to reliably sort elements alphabetically.

```
    const arr2 = s2.split("").sort();
```

Provide a compare function that depends on "String.localeCompare", to reliably sort elements alphabetically.

Fig. 6. Violation of `javascript:S2871`: `Array.prototype.sort()` should use a compare function

5.3 RQ3: Analysis of Maintainability

Code smells often refer to issues that don't cause immediate program failures, but could make the code difficult to maintain or evolve. Therefore, we analyze whether and to what extent code smells appear in code suggestions generated by the LLM-based tools. For maintainability analysis, we first excluded structural smell issues reflecting

Just Accepted

```
let maxVal = -Infinity;
for (let i = 0; i < n.length + 1; i++) {
    const newVal = parseInt(n.slice(0, i) + x + n.slice(i));
    if (isNegative) {
        newVal *= -1;
```

Correct this attempt to modify "newVal" or use "let" in its declaration.

Fig. 7. Violation of `javascript:S3500`: `const` variables should not be reassigned

file-level attributes or naming conventions rather than generation quality, including cxx:FileHeader, cxx:FileName, cxx:TooLongLine, cxx:FunctionName, cxx:MethodName, python:S100, and java:S1220. After this adjustment, we conducted a sampled manual validation of the remaining code smell issues. The results showed that 62.7% of the sampled issues were confirmed (95% CI: [57.1%, 67.9%]), and 11.0% were False Positives. These findings indicate that the impact of false positives is acceptable and that the static analysis results provide a reliable foundation for comparing maintainability across tools and languages.

As shown in Figure 8, the grouped-and-stacked bar chart reports the average number of code smells per file across tools and difficulty levels (Easy, Medium, Hard), with each bar stacked by programming language (C, C++, Java, JavaScript, Python). For example, in Tongyi Lingma's code suggestions for Easy problems, an average of 1.32 code smells per code suggestion is observed in C, 1.23 in C++, 1.70 in Java, 0.62 in JavaScript, and 0.19 in Python.

For the tool-specific analysis, Tongyi Lingma exhibits the lowest maintainability performance. It has the highest number of code smells among all tools, with an average of 1.46 code smells per code suggestion. CodeWhisperer and Copilot follow closely, with averages of 1.43 and 1.42 code smells per suggestion, respectively. Codeium and Tabnine generate fewer code smells, with averages of 1.34 and 1.27, indicating that they can produce cleaner and more maintainable code suggestions.

For the cross-language analysis, we observe that code smells are almost ubiquitous in C, C++, and Java code suggestions, with nearly every file containing at least one issue. This prevalence reflects the inherent verbosity and stricter syntax requirements of statically typed languages. Among these languages, Java exhibits the worst maintainability, with an average of 3.15 code smells per code suggestion, which is higher than both C and C++. JavaScript and Python show a sparser distribution of code smells, with less than one-third of the suggestions being affected on average. This suggests that code generation tools tend to produce cleaner outputs in dynamically typed languages, because they have simpler syntax and more flexible semantics.

For tool-language combination analysis, all tools in Java tend to produce a higher number of smells, but Copilot-Java and CodeWhisperer-Java rank worst. They consistently generated code with multiple unused assignments and poor variable management. These combinations need mandatory post-generation refactoring. Tongyi Lingma-C++ produces more smells than others, particularly for statement-level issues, suggesting a less refined understanding of C++ idioms. On the other hand, several combinations show clean code generation. Both Tabnine-C and Tabnine-C++ have fewer structural smells than others, showing a better handling of low-level language idioms. Codeium-JavaScript generates the lowest smell counts in JavaScript. Tongyi Lingma-Java produces fewer code smells, making it a better choice for Java maintainability.

From the perspective of smell types, we found that all code suggestions in C and C++ were flagged for the **cxx:TabCharacter** code smell. That is, tabulation characters should not be used. Since tab display width varies from editor to editor, this can lead to inconsistent code indentation in different development environments, thereby reducing readability. Besides, **cxx:ParsingErrorRecovery** and **cxx:TooManyStatementsPerLine** are frequent code smells. Together, they account for more than 35% of all cases. The **ParsingErrorRecovery** smell typically

Just Accepted

arises when the code structure contains irregular or incomplete syntactic elements. **TooManyStatementsPerLine** occurs when several statements are written on one line, which reduces readability.

In Java, the two most frequently occurring code smells are **java:S1854** (*Unused assignments should be removed*) and **java:S1659** (*Multiple variables should not be declared on the same line*). **S1854** appears in 86.61% of all cases. It usually occurs in unused variable assignments during initialization that don't affect program logic. **S1659** accounts for 6.79%. It is typically found when multiple variables are declared on the same line, which reduces readability. The high frequency of these two smells indicates that current models tend to produce Java code with unclear control flow and ambiguous variable usage. These issues reduce the overall readability and modifiability of the code, increase the cost of understanding and testing, and pose potential risks to future maintenance.

In JavaScript, the most frequently occurred code smell is **javascript:S4138** (*"for of" should be used with Iterables*), which accounts for 31.37% of all detected issues. This code smell happens when a traditional `for` loop is used to iterate over iterable objects, where a more concise and readable `for...of` loop is expected. The `for...of` construct not only improves readability but also reduces the likelihood of common mistakes, such as *off-by-one errors*. The repeated presence of this issue suggests that LLMs tend to use lower-level loop structures, since the frequent occurrence of such patterns in the training data. In addition, **javascript:S3776** (*Cognitive Complexity of functions should not be too high*) and **javascript:S3504** (*Variables should be declared with "let" or "const"*) are also common, accounting for 15.98% and 14.23% of all issues, respectively. `S3776` appears in functions with high cognitive complexity, often due to nested loops and conditional statements. `S3504` occurs when variables are declared with `var` instead of `let` or `const`, which reduces code safety and clarity.

In Python, the most prevalent code smell is **python:S117** (*Local variable and function parameter names should comply with a naming convention*), which accounts for 49.84% of all detected issues. Although these violations do not affect functional correctness, they can negatively impact code readability and long-term maintenance, especially in collaborative or large-scale projects. The prevalence of this issue reflects the fact that LLMs often inherit inconsistent naming styles from their training data, which may include a mix of formal codebases and informal examples. In addition, **python:S3776** (*Cognitive Complexity of functions should not be too high*) is the second most frequent smell, accounting for 22.15%. It appears in functions with deeply nested loops or long conditional chains, which make the logic hard to follow.

In the difficulty-level analysis, hard problems tend to contain more code smells per file. This pattern is consistent across most tools and languages, suggesting that as task complexity increases, the likelihood of generating structurally flawed code rises. This observation aligns with our earlier findings on correctness and reliability, further suggesting that LLM-based code generation tools struggle to maintain quality under more complex problem settings.

5.4 RQ4: Analysis of Code Complexity

Code complexity directly impacts the effort needed for comprehension and maintenance in real-world development. When the control flow becomes intricate, developers need to track multiple execution paths while reading, debugging, and modifying code. This increases the likelihood of introducing errors during bug fixes or feature additions. Therefore, complexity represents an important dimension of code quality for generated code. In our study, we use cyclomatic complexity as the measurement metric, which quantifies the number of independent execution paths through code. Each conditional check or branching construct increments the complexity score. Higher cyclomatic complexity indicates more branches and conditional statements, which in turn requires more test cases to achieve path coverage and more cognitive effort to understand program behavior. Code with high complexity is generally difficult to read, test, and maintain. This makes cyclomatic complexity particularly relevant for evaluating AI-generated code, as developers may not fully comprehend the suggestions when accepting them.

Just Accepted

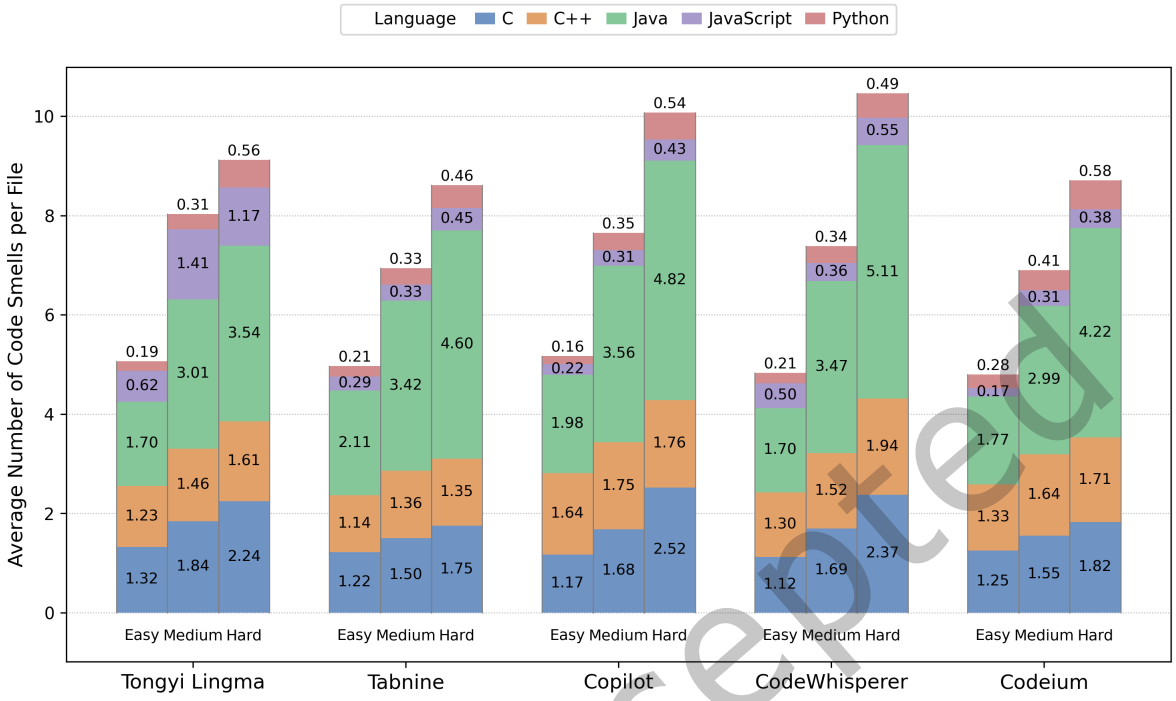

Fig. 8. Average number of code smells per code suggestion across programming languages and difficulty levels

Table 3 presents the cyclomatic complexity of code suggestions generated by five tools across five programming languages. The lower quartile (LQ) marks the value at the 25th percentile, and the upper quartile (UQ) marks the value at the 75th percentile. The median (MED) corresponds to the value at the 50th percentile. Additionally, we report the number of outliers with high cyclomatic complexity. Following McCabe's definition and guidelines [45], we consider the files with a cyclomatic complexity greater than 20 as outliers. Figure 9 shows the example of an outlier. The code repeatedly employs nested loop structures with similar conditional checks. This substantially increases the number of independent paths and leads to high cyclomatic complexity.

From the table, we can see that most of the code suggestions don't have high complexity. The median cyclomatic complexity across all tool-language combinations ranges from 4 to 6, indicating that the generated code typically contains a moderate number of decision points and control flow branches. This suggests that current LLM-based code generation tools generally produce algorithmically straightforward solutions rather than overly complex nested structures.

For the tool-specific analysis, Tabnine demonstrates the most consistent performance in complexity control, with the lowest number of outliers across most programming languages. Its median complexity values remain stable, typically ranging from 4 to 5. This suggests Tabnine can generate straightforward, structurally simple implementations. However, it shows lower functional correctness in RQ1. This inverse relationship indicates that Tabnine's structural simplicity comes at the cost of algorithmic sophistication. It may prioritize simple implementations but often fails to handle complex logic correctly. CodeWhisperer and GitHub Copilot show higher cyclomatic complexity. For example, CodeWhisperer produces 39 outliers in C++ and 17 outliers in JavaScript, and GitHub Copilot produces 52 outliers in C. Although these two tools achieve better functional correctness

Just Accepted

Table 3. Cyclomatic Complexity Indicators of Code Generated by Different Tools Across Programming Languages

Language	Metric	Tongyi Lingma	Tabnine	Copilot	CodeWhisperer	Codeium
C	LQ	4	3	4	4	4
	MED	5	5	6	6	5
	UQ	8	7	8	8	7
	#Outliers	82	22	52	39	15
C++	LQ	3	3	3	4	3
	MED	5	4	5	5	5
	UQ	6	6	7	8	7
	#Outliers	14	9	13	32	10
Java	LQ	3	3	4	4	3
	MED	5	5	5	5	5
	UQ	6	6	7	7	7
	#Outliers	6	5	7	10	17
JavaScript	LQ	4	4	4	4	4
	MED	5	5	5	5	5
	UQ	7	7	7	8	7
	#Outliers	10	8	11	17	17
Python	LQ	2	3	3	3	3
	MED	4	4	4	4	4
	UQ	5	6	7	6	6
	#Outliers	11	5	8	5	4

(as shown in RQ1), they sometimes generate more complex control structures. For short-term tasks, both tools can effectively generate complex but correct code that passes test cases. However, they may pose challenges for projects requiring long-term maintenance. High complexity could increase the cognitive burden during code reviews, complicate debugging, and elevate the difficulty of future modifications. The initial productivity gains from accepting correct suggestions are beneficial, but the cumulative maintenance costs over the code's lifecycle may diminish or eliminate this advantage. Codeium presents moderate performance. Compared with tools that produce higher complexity, it may be slightly weaker in correctness. However, its generated code tends to be simpler and easier to understand. Tongyi Lingma exhibits the highest variability in complexity, with 82 outliers in C. This indicates inconsistent complexity control, potentially reflecting less robust training on diverse algorithmic patterns.

These observations reveal different trade-off patterns between correctness and complexity across tools. Some tools tend to consider more complex implementations to achieve higher pass rates. Other tools tend to generate relatively simple code. Although the LeetCode dataset consists of small solutions, we can still observe meaningful differences in cyclomatic complexity. If similar generation patterns appear in larger projects, these differences are likely to grow and have a stronger impact on debugging and feature evolution. For practitioners, tool selection should not only focus on short-term correctness, but also consider the project lifecycle and maintenance costs induced by higher complexity. For rapid prototyping and exploratory development, CodeWhisperer or Copilot

Just Accepted

may be feasible options. Their complex but correct code facilitates faster initial progress. The complexity concerns can be addressed through later refactoring. But for software projects with long maintenance lifecycles, code will be read, debugged, and modified repeatedly over the years. Therefore, prioritizing code clarity and simplicity is important. In such cases, accepting Tabnine's simpler implementations and investing time in manual correctness refinement may obtain better long-term outcomes. Tools with balanced profiles, such as Codeium, may also be preferable despite lower peak correctness, as it offers a middle option between immediate functionality and sustained maintainability.

For the language-specific analysis, C demonstrates the highest complexity burden across all tools, with consistently higher median values (5-6) and the most outliers. This reflects the inherent complexity of low-level programming, where manual memory management and pointer arithmetic often require additional conditional checks and error handling. The high number of outliers in C (particularly 82 for Tongyi Lingma) suggests that models struggle to generate efficient, simple solutions for systems programming tasks. Python exhibits the lowest complexity across all metrics, with median values of 4 and relatively few outliers. This aligns with Python's design philosophy of readability and simplicity, and suggests that models can more effectively leverage Python's high-level constructs to produce concise solutions. C++, Java, and JavaScript show intermediate complexity levels, with median values consistently around 5. However, the distribution of outliers varies noticeably across tools, indicating that model-specific training approaches have different impacts on complexity management in these languages.

For the tool-language combination analysis, the median and quartiles of cyclomatic complexity are similar across different tools. For most combinations, the median values range from 4 to 6. However, some combinations generate a greater number of high-complexity files than others. For example, Tongyi Lingma produces more outliers in C than the other tools, and CodeWhisperer also reports a relatively high number of outliers in C++. That is, most of the code suggestions by Tongyi Lingma-C and CodeWhisperer-C++ exhibit a complexity similar to those of other combinations. However, they produce more outliers that show higher cyclomatic complexity and need extra attention in maintenance.

6 Discussion

In this section, we discuss our findings and their implications.

Finding 1. Our evaluation reveals that functional correctness remains the most significant barrier to widespread adoption of LLM-based code generation tools. The best-performing tool, CodeWhisperer, achieves only 57.56% average correctness across all languages, while most tools perform even worse. These results indicate a fundamental limitation in current approaches. The dominance of Wrong Answer (WA) errors, which account for 50-80% of all failures, further highlights this issue. It suggests that although LLM-based tools have largely mastered the syntactic aspects of programming languages, they struggle with algorithmic reasoning and semantic correctness. Although the tools can generate code that compiles and runs without crashing, they fail to implement the intended logic correctly. This finding has critical implications for deployment strategies, indicating that current tools are better suited as coding assistants that require human oversight rather than autonomous code generators. Meanwhile, the inverse relationship between problem difficulty and correctness across all tools indicates a fundamental scaling challenge. As algorithmic complexity increases, the code generation tools' ability to maintain logical coherence decreases markedly. This pattern suggests that current training approaches may be insufficient for handling complex reasoning tasks that require multi-step logical deduction and intricate state management.

Practitioners should implement tiered deployment strategies that use these tools for routine coding tasks with standard code review. For more complex problems, intensive peer reviews and comprehensive test coverage are necessary. Organizations should establish clear guidelines that define complexity thresholds triggering enhanced

Just Accepted

```
int numberOfGoodSubarraySplits(int* nums, int numsSize){
    int i,j,k,l,m,n,o,p,q,r,s,t,u,v,w,x,y,z,a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,v,w,x,y,z,A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P = 0;
    for(i=0;i<numsSize;i++){
        if(nums[i]==1){
            for(j=i+1;j<numsSize;j++){
                if(nums[j]==1){
                    for(k=j+1;k<numsSize;k++){
                        if(nums[k]==1){
                            for(l=k+1;l<numsSize;l++){
                                if(nums[l]==1){
                                    for(m=l+1;m<numsSize;m++){
                                        if(nums[m]==1){
                                            for(n=m+1;n<numsSize;n++){
                                                if(nums[n]==1){
                                                    for(o=n+1;o<numsSize;o++){
                                                        if(nums[o]==1){
                                                            for(p=o+1;p<numsSize;p++){
                                                                if(nums[p]==1){
                                                                    for(q=p+1;q<numsSize;q++){
                                                                        if(nums[q]==1){
                                                                            for(r=q+1;r<numsSize;r++){
                                                                                if(nums[r]==1){
                                                                                    for(s=r+1;s<numsSize;s++){
                                                                                        if(nums[s]==1){
                                                                                            for(t=s+1;t<numsSize;t++){
                                                                                                if(nums[t]==1){
                                                                                                    for(u=t+1;u<numsSize;u++){
                                                                                                        if(nums[u]==1){
                                                                                                            for(v=u+1;v<numsSize;v++){
                                                                                                                if(nums[v]==1){
                                                                                                                    for(w=v+1;w<numsSize;w++){
                                                                                                                        if(nums[w]==1){
                                                                                                                            for(x=w+1;x<numsSize;x++){
                                                                                                                                if(nums[x]==1){
                                                                                                                                    for(y=x+1;y<numsSize;y++){
                                                                                                                                        if(nums[y]==1){
                                                                                                                                    return 1;}}}
```

datasets that adequately represent lower-level languages. It is also beneficial to explore transfer learning techniques that use knowledge from high-resource languages to improve performance in low-resource languages. Additionally, there is a need to develop specialized fine-tuning approaches for handling error-prone language features such as pointer arithmetic and memory management.

Finding 3. Our analysis reveals a complex relationship between different dimensions of code quality, which challenges simple assumptions about what defines quality. While some tools, like Codeium, excel in reliability by producing fewer buggy code suggestions, others, like CodeWhisperer, achieve higher functional correctness but with increased complexity. This trade-off suggests that different tools may be optimized for different aspects of code quality, which poses a demand for users to make strategic decisions based on their specific priorities. Meanwhile, the concentration of bugs in Medium-difficulty problems is particularly noteworthy, as these represent the common complexity level in real-world development. Even for Easy problems, none of the tools can ensure 100% bug-free code suggestions. This indicates that current tools still require continuous human oversight, regardless of task complexity.

Selecting tools should align with project-specific quality priorities. For example, Codeium for reliability-critical applications where minimizing bugs is important, CodeWhisperer when functional correctness outweighs other concerns, and Tabnine when consistent complexity control is valued. Development teams should conduct pilot evaluations to determine which tool best matches their domain requirements and quality standards. Future research could focus on multi-objective optimization frameworks that enhance functional correctness, reliability, and maintainability during code generation. Additionally, researchers could develop hybrid approaches that combine LLM generation with symbolic execution to guarantee specific correctness properties.

Finding 4. The prevalence of code smells across all tools, especially in statically typed languages, raises concerns about the long-term maintainability of generated code. Notably, Java exhibits the worst maintainability (3.15 code smells per code suggestion), despite being a mature, well-structured language. This suggests that LLM-based tools may prioritize functional implementation over structural quality. Additionally, the widespread presence of formatting and style issues, such as the frequent cxx:TabCharacter violations in C/C++, indicates that current models lack a consistent understanding of best practices and coding standards. These finding have profound implications for enterprise adoption, where code consistency and maintainability are critical for long-term project success.

Development workflows should integrate automated static analysis tools into CI/CD pipelines. This will help developers automatically identify code smells in AI-generated code before merge. For Java projects, it is essential to establish mandatory post-generation refactoring protocols to address prevalent issues, such as unused variable assignments (java:S1854) and multiple variable declarations per line (java:S1659). Additionally, configure IDE plugins to automatically apply language-specific formatting standards and linting rules to generated code upon acceptance. Future research could focus on quality-aware training objectives that explicitly balance functional correctness with structural quality metrics. This involves developing specialized post-generation refinement models trained to eliminate common smell patterns. Moreover, exploring style-aware training approaches can promote clean coding principles and best practices.

Our findings indicate that current LLM-based code generation tools are most appropriately considered as intelligent coding assistants rather than autonomous programmers. The moderate rates of functional correctness and structural quality issues highlight the need for human oversight throughout the development process. This perspective shifts the focus from "AI replacing programmers" to "AI enhancing programmer productivity". The tools present greater effectiveness for routine coding tasks, and provide initial implementation sketches that developers can refine and optimize. For complex algorithmic challenges or systems programming tasks, human expertise remains indispensable.

Furthermore, the variability in performance across tools, languages, and problem difficulties highlights sophisticated risk assessment frameworks. Development teams adopting these tools must implement robust testing

Just Accepted

and code review processes, particularly for Medium and Hard complexity tasks where reliability issues are most prevalent. The widespread occurrence of structural quality problems suggests that static analysis tools and automated quality checks become even more critical when incorporating AI-generated code into development workflows. Development teams must also consider the long-term effects of using AI-generated code that may have maintenance issues. Traditional quality assurance processes may require enhancement to address the specific failure modes and quality patterns exhibited by LLM-generated code. This is important to ensure that short-term productivity gains do not compromise long-term code quality.

Finally, our comprehensive evaluation reveals that no single tool consistently excels across all quality dimensions and programming languages. Instead, each tool presents distinct trade-offs that make it suitable for different development contexts and priorities. In most cases, CodeWhisperer and GitHub Copilot achieve higher functional correctness. CodeWhisperer performs particularly well in statically typed languages such as C++ and Java. However, these improvements in correctness often come with higher maintenance costs. Both tools generate more code smells and higher cyclomatic complexity. This presents challenges for code review, testing, and long-term maintenance due to intricate control structures and structural quality issues. In contrast, Tabnine consistently shows low complexity across languages. This suggests that it tends to generate code with simpler control flow. But Tabnine has the lowest functional correctness and introduces many bugs. Thus, it is better suited for generating initial code skeletons or partial implementations that developers will manually refine and debug, rather than as a source of reliable complete solutions. Tongyi Lingma and Codeium are in middle positions with language-specific strengths. Codeium achieves moderate functional correctness but excels in reliability. It produces the fewest bugs across most languages, particularly in Java and Python. This makes Codeium well-suited for software systems where stability and bug avoidance are prioritized over peak correctness. Tongyi Lingma generates Java code with slightly fewer code smells than the other tools. However, both tools exhibit inconsistent performance across languages. They may be more appropriate for their strong language contexts (e.g., Codeium for Java/Python reliability, Tongyi Lingma for Java maintainability). Therefore, tool selection should be driven by explicit prioritization of quality dimensions aligned with project requirements. Development teams need to evaluate tools based on their specific language ecosystem, quality priorities, development phase, and quality assurance capacity, rather than seeking universally optimal solutions.

7 Threats to Validity

First, the experimental data in this paper come from the LeetCode platform, whose task types are mainly algorithmic questions. It may not fully reflect the characteristics of real industrial projects, including the interaction between multiple modules, complex engineering constraints, a multi-person collaborative development process, and long-term maintenance requirements for large projects. In addition, although the programming languages involved in our experiments cover the mainstream C, C++, Java, Python, and JavaScript, they still do not represent all the language features and development environments used in industrial practice. In particular, we didn't analyze other languages that are increasingly used in industrial projects, such as Go, Rust, etc, thus there still exist limitations in the language generalization of our experimental conclusions.

Second, our evaluation uses small and self-contained LeetCode problems rather than medium or large codebases. This may pose a limitation, but it enables our core contribution: systematic comparative evaluations across tools, languages, and quality dimensions. Using small and controlled problems can provide essential advantages for comparative research: 1) Identical inputs for all tools avoid confounding variables from project-specific context; 2) LeetCode's cross-language problem availability enables direct comparison between languages; 3) Automated testing provides objective correctness evaluation for 65,325 suggestions; and 4) The methodology is fully reproducible using public infrastructure. This approach can effectively evaluate tools' intrinsic generation capabilities. We believe that our comparative findings could generalize to larger contexts, as they reflect fundamental tool

Just Accepted

differences. However, evaluating medium- to large-scale codebases could provide additional insights about context handling, long-range dependency handling, architectural coherence, and incremental development patterns. Therefore, future work that examines code suggestion tools in medium or large scenarios would complement our comparative evaluation effectively.

Third, our evaluation doesn't include a dedicated security vulnerability assessment. In this study, we focus on evaluating code quality across four dimensions: functional correctness, reliability, maintainability, and complexity. Some defect types identified in RQ2, such as null pointer dereferences and integer overflows, could evolve into security vulnerabilities in real projects under specific inputs or attack scenarios. However, since our experiments were conducted in LeetCode's closed testing environment without modeling explicit threat scenarios, we treated these issues uniformly as reliability-related bugs rather than strictly classifying them as security vulnerabilities. Future work should extend our evaluation framework to incorporate security-oriented benchmarks and explicit threat models, which enable a more detailed quantitative analysis of potential vulnerability risks. This could include employing specialized security analysis tools (such as CodeQL, Semgrep) and applying vulnerability classification schemes (e.g., CVE) to systematically assess the prevalence of security issues across different tools, languages, and vulnerability categories. Such extensions would provide a more complete picture of code generation risks and help compensate for the current study's limitations in the security dimension.

Forth, our research leverages SonarQube and Cppcheck for code quality analysis, which may bring limitations. Since static analysis tools may produce false positives, this potentially affects the accuracy of our measurement. To mitigate this threat, we conducted manual validation of sampled issues. The results demonstrated that tool findings can fall within acceptable precision ranges for comparative analysis. However, our validation examined only a sample of identified issues. Therefore, measurement errors introduced by static analysis tools may still influence our conclusions to some extent. But we believe the impact on cross-tool and cross-language comparisons remains limited.

Finally, the model selection for our evaluation is the five current representative code generation tools, and does not cover all the newly developed large language models or updated versions. In addition, the training data, model versions, and parameter settings of each code generation model are continuously evolving. As a result, the conclusions drawn from our experiments may change with future model updates or to training methodologies. This may pose an external threat to validity.

8 Conclusion

In this paper, we conduct a comprehensive study of code quality in LLM-based code generation tools. We analyze five mainstream tools across five programming languages using 2,613 LeetCode problems. Our evaluations cover functional correctness, reliability, maintainability, and complexity.

Through our analysis of 65,325 code suggestions, we have presented that: 1) Functional correctness could be the primary barrier to widespread adoption, with the best-performing tool achieving only 57.56% average correctness. The dominance of Wrong Answer errors (50-80% of failures) indicates that while LLMs handle syntax adequately, they struggle with algorithmic reasoning and semantic correctness. 2) Programming language characteristics significantly influence quality. Python and JavaScript demonstrate superior performance (41.91% and 41.75% respectively), while C presents the greatest challenge (25.66%), highlighting limitations in low-level programming tasks. 3) Reliability presents additional concerns, with bugs occurring more frequently in medium-difficulty tasks. Maintainability concerns are pervasive, particularly in statically typed languages where code smells are nearly ubiquitous. Although complexity is generally well-controlled, there are still outliers with excessive complexity across all tools. 4) Different tools show distinct behaviors under different languages. CodeWhisperer and GitHub Copilot achieve relatively high functional correctness, but they also tend to produce more bugs or code smells. Tabnine generates code with low cyclomatic complexity and fewer code smells, but it produces many bugs and its

Just Accepted

functional correctness is the lowest among the five tools. Tongyi Lingma and Codeium lie in the middle overall and show relative strengths in specific languages.

These findings indicate LLM-based tools are intelligent coding assistants rather than autonomous programmers. Current tools are most effective for routine tasks, but they require continuous human oversight. Tool selection should depend on the usage context. For short-term programming tasks, developers can use tool-language combinations with higher correctness, as long as the code is thoroughly tested. For long-running components, they should avoid combinations with low quality. Organizations adopting these tools must balance short-term productivity gains against long-term code quality considerations. In addition, they need to implement robust testing frameworks tailored to AI-generated code, with enhanced quality assurance processes addressing specific LLM failure modes.

Acknowledge

This work is supported by the Open Research Fund of the State Key Lab. for Novel Software Technology (Nanjing University) under grant No. KFKT2025B08.

References

[1] Uri Alon, Roy Sadaka, Omer Levy, and Eran Yahav. 2020. Structural language models of code. In *International conference on machine learning*. PMLR, 245–256.

[2] Anthropic. 2024. The Claude 3 Model Family: Opus, Sonnet, Haiku. https://www.anthropic.com/news/claude-3-opus-sonnet-haiku. Model card.

[3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).

[4] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[6] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, et al. 2022. Multipl-e: A scalable and extensible approach to benchmarking neural code generation. *arXiv preprint arXiv:2208.08227* (2022).

[7] Liguo Chen, Qi Guo, Hongrui Jia, Zhengran Zeng, Xin Wang, Yijiang Xu, Jian Wu, Yidong Wang, Qing Gao, Jindong Wang, et al. 2024. A survey on evaluating large language models in code generation tasks. *arXiv preprint arXiv:2408.16498* (2024).

[8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[9] Matteo Ciniselli, Nathan Cooper, Luca Pascarella, Denys Poshyvanyk, Massimiliano Di Penta, and Gabriele Bavota. 2021. An empirical study on the usage of bert models for code completion. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 108–119.

[10] Trevor Cohn, Phil Blunsom, and Sharon Goldwater. 2010. Inducing tree-substitution grammars. *The Journal of Machine Learning Research* 11 (2010), 3053–3096.

[11] Tristan Coignion, Clément Quinton, and Romain Rouvoy. 2024. A performance study of llm-generated code on leetcode. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 79–89.

[12] Domenico Cotroneo, Alessio Foggia, Cristina Improta, Pietro Liguori, and Roberto Natella. 2024. Automating the correctness assessment of AI-generated code for security contexts. *Journal of Systems and Software* 216 (2024), 112113.

[13] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.

[14] Ankur Desai and Atul Deo. 2022. Introducing Amazon CodeWhisperer, the ML-powered coding companion. https://aws.amazon.com/blogs/machine-learning/introducing-amazon-codewhisperer-the-ml-powered-coding-companion/. Accessed: 2025-06-02.

[15] Yihong Dong, Jiazheng Ding, Xue Jiang, Ge Li, Zhuo Li, and Zhi Jin. 2025. Codescore: Evaluating code generation by learning code execution. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–22.

[16] Mingzhe Du, Anh Tuan Luu, Bin Ji, Qian Liu, and See-Kiong Ng. 2024. Mercury: A code efficiency benchmark for code large language models. *arXiv preprint arXiv:2402.07844* (2024).

Just Accepted

[17] Akiko Eriguchi, Kazuma Hashimoto, and Yoshimasa Tsuruoka. 2016. Tree-to-sequence attentional neural machine translation. *arXiv preprint arXiv:1603.06075* (2016).
[18] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
[19] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
[20] Sumit Gulwani. 2010. Dimensions in program synthesis. In *Proceedings of the 12th international ACM SIGPLAN symposium on Principles and practice of declarative programming*. 13–24.
[21] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming–The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
[22] Abram Hindle, Earl T Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the naturalness of software. *Commun. ACM* 59, 5 (2016), 122–131.
[23] Dong Huang, Yuhao Qing, Weiyi Shang, Heming Cui, and Jie Zhang. 2024. Effibench: Benchmarking the efficiency of automatically generated code. *Advances in Neural Information Processing Systems* 37 (2024), 11506–11544.
[24] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2016. Summarizing source code using a neural attention model. In *54th Annual Meeting of the Association for Computational Linguistics 2016*. Association for Computational Linguistics, 2073–2083.
[25] Susmit Jha, Sumit Gulwani, Sanjit A Seshia, and Ashish Tiwari. 2010. Oracle-guided component-based program synthesis. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 1*. 215–224.
[26] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515* (2024).
[27] Aravind Joshi and Owen Rambow. 2003. A formalism for dependency grammar based on tree adjoining grammar. In *Proceedings of the Conference on Meaning-text Theory*. MTT Paris, France, 207–216.
[28] Muhammad Fawad Akbar Khan, Max Ramsdell, Erik Falor, and Hamid Karimi. 2023. Assessing the promise and pitfalls of chatgpt for automated code generation. *arXiv preprint arXiv:2311.02640* (2023).
[29] Seohyun Kim, Jinman Zhao, Yuchi Tian, and Satish Chandra. 2021. Code prediction by feeding trees to transformers. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 150–162.
[30] LeetCode. [n. d.]. LeetCode: Online Coding Platform. https://leetcode.com/. Accessed: 2025-06-02.
[31] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).
[32] Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D Le, and David Lo. 2024. Refining chatgpt-generated code: Characterizing and mitigating code quality issues. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024), 1–26.
[33] Zhiqiang Liu, Yong Dou, Jingfei Jiang, and Jinwei Xu. 2016. Automatic code generation of convolutional neural networks in FPGA implementation. In *2016 International conference on field-programmable technology (FPT)*. IEEE, 61–68.
[34] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173* (2024).
[35] Zeyuan Ma, Hongshu Guo, Jiacheng Chen, Guojun Peng, Zhiguang Cao, Yining Ma, and Yue-Jiao Gong. 2024. LLaMoCo: Instruction tuning of large language models for optimization code generation. *arXiv preprint arXiv:2403.01131* (2024).
[36] Ran Mo, Dongyu Wang, Wenjing Zhan, Yingjie Jiang, Yepeng Wang, Yuqi Zhao, Zengyang Li, and Yutao Ma. [n. d.]. Assessing and Analyzing the Correctness of GitHub Copilot's Code Suggestions. *ACM Transactions on Software Engineering and Methodology* ([n. d.]).
[37] Michael Alexander Florez Muñoz, Juan Camilo Jaramillo De La Torre, Stefany Pareja López, Stiven Herrera, and Christian Andrés Candela Uribe. 2024. Comparative Study of AI Code Generation Tools: Quality Assessment and Performance Analysis. *LatIA* 2 (2024), 21.
[38] Tung Thanh Nguyen, Anh Tuan Nguyen, Hoan Anh Nguyen, and Tien N Nguyen. 2013. A statistical semantic language model for source code. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. 532–542.
[39] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022).
[40] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
[41] Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, and Andrea De Lucia. 2014. Do they really smell bad? a study on developers' perception of bad code smells. In *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 101–110.
[42] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the keyboard? assessing the security of github copilot's code contributions. *Commun. ACM* 68, 2 (2025), 96–105.

Just Accepted

[43] Veselin Raychev, Martin Vechev, and Eran Yahav. 2014. Code completion with statistical language models. In *Proceedings of the 35th ACM SIGPLAN conference on programming language design and implementation*. 419–428.
[44] Xinyu She, Yue Liu, Yanjie Zhao, Yiling He, Li Li, Chakkrit Tantithamthavorn, Zhan Qin, and Haoyu Wang. 2023. Pitfalls in language models for code intelligence: A taxonomy and survey. *arXiv preprint arXiv:2310.17903* (2023).
[45] V Siahaan, H Ginting, and M Amri. 2025. Cyclomatic Complexity and Maintainability in Modern Software: A Systematic Review. *Journal of Data Science, Technology, and Computer Science* 5, 1 (2025), 8–16.
[46] Mohammed Latif Siddiq, Beatrice Casey, and Joanna CS Santos. 2024. FRANC: A Lightweight Framework for High-Quality Code Generation. In *2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 106–117.
[47] Da Song, Zijie Zhou, Zhijie Wang, Yuheng Huang, Shengmai Chen, Bonan Kou, Lei Ma, and Tianyi Zhang. 2023. An empirical study of code generation errors made by large language models. In *7th Annual Symposium on Machine Programming*.
[48] Ioannis Stamelos, Lefteris Angelis, Apostolos Oikonomou, and Georgios L Bleris. 2002. Code quality analysis in open source software development. *Information systems journal* 12, 1 (2002), 43–60.
[49] Zeyu Sun, Qihao Zhu, Lili Mou, Yingfei Xiong, Ge Li, and Lu Zhang. 2019. A grammar-based structural cnn decoder for code generation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 7055–7062.
[50] Tabnine. [n. d.]. Tabnine's Private and Protected Universal Models. https://docs.tabnine.com/main/welcome/readme/ai-models/tabnines-private-and-protected-universal-models. Accessed: 2025-06-02.
[51] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).
[52] TIOBE Software BV. [n. d.]. TIOBE Index for September 2025. https://www.tiobe.com/tiobe-index/. Accessed: 2025-09-04.
[53] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
[54] Jonas F Tuttle, Dayuan Chen, Amina Nasrin, Noe Soto, and Ziliang Zong. 2024. Can LLMs Generate Green Code-A Comprehensive Study Through LeetCode. In *2024 IEEE 15th International Green and Sustainable Computing Conference (IGSC)*. IEEE, 39–44.
[55] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
[56] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S Yu. 2018. Improving automatic source code summarization via deep reinforcement learning. In *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*. 397–407.
[57] Dongyu Wang, Ran Mo, Wenjing Zhan, and Yingjie Jiang. 2024. An Analysis of the Code Quality Generated by GitHub Copilot. *Jisuanji Kexue (Computer Science)* (2024), 1–23.
[58] Jiexin Wang, Xitong Luo, Liuwen Cao, Hongkui He, Hailin Huang, Jiayuan Xie, Adam Jatowt, and Yi Cai. 2024. Is your ai-generated code really safe? evaluating large language models on secure code generation with codeseceval. *arXiv preprint arXiv:2407.02395* (2024).
[59] Lun Wang, Chuanqi Shi, Shaoshuai Du, Yiyi Tao, Yixian Shen, Hang Zheng, and Xinyu Qiu. 2024. Performance Review on LLM for solving leetcode problems. In *2024 4th International Symposium on Artificial Intelligence and Intelligent Manufacturing (AIIM)*. IEEE, 1050–1054.
[60] Xin Wang, Yasheng Wang, Yao Wan, Fei Mi, Yitong Li, Pingyi Zhou, Jin Liu, Hao Wu, Xin Jiang, and Qun Liu. [n. d.]. Compilable neural code generation with compiler feedback, 2022. *URL https://arxiv. org/abs/2203.05132* ([n. d.]).
[61] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, and Steven CH Hoi. 2023. Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922* (2023).
[62] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021).
[63] Windsurf. [n. d.]. Getting Started with Windsurf. https://docs.windsurf.com/windsurf/getting-started. Accessed: 2025-06-02.
[64] Burak Yetiştiren, Işık Özsoy, Miray Ayerdem, and Eray Tüzün. 2023. Evaluating the code quality of ai-assisted code generation tools: An empirical study on github copilot, amazon codewhisperer, and chatgpt. *arXiv preprint arXiv:2304.10778* (2023).
[65] Burak Yetistiren, Isik Ozsoy, and Eray Tuzun. 2022. Assessing the quality of GitHub copilot's code generation. In *Proceedings of the 18th international conference on predictive models and data analytics in software engineering*. 62–71.
[66] Xiao Yu, Lei Liu, Xing Hu, Jacky Wai Keung, Jin Liu, and Xin Xia. 2024. Fight fire with fire: How much can we trust chatgpt on source code-related tasks? *IEEE Transactions on Software Engineering* (2024).
[67] Xiao Yu, Lei Liu, Xing Hu, Jacky Wai Keung, Jin Liu, and Xin Xia. 2024. Where Are Large Language Models for Code Generation on GitHub? *arXiv preprint arXiv:2406.19544* (2024).
[68] Shuyan Zhou, Uri Alon, Sumit Agarwal, and Graham Neubig. 2023. Codebertscore: Evaluating code generation with pretrained models of code. *arXiv preprint arXiv:2302.05527* (2023).

Just Accepted